

Analyzing Cryptographic Protocols with Tamarin



Fifteenth SRI Summer School on Formal Techniques, Menlo Park, 2026



Cas Cremers



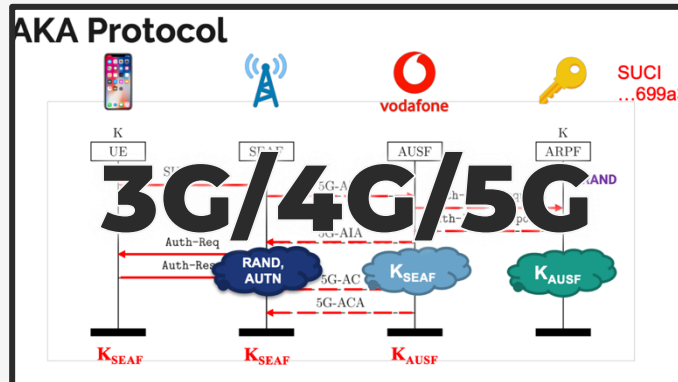


Preparing for the lab session

- Check the lab session files here:
<https://github.com/cascremers/tamarin-tutorial-ssft2026>
- Install Tamarin *before* the lab session
- Main reference: the Tamarin Book
available at <http://www.tamarin-prover.com>

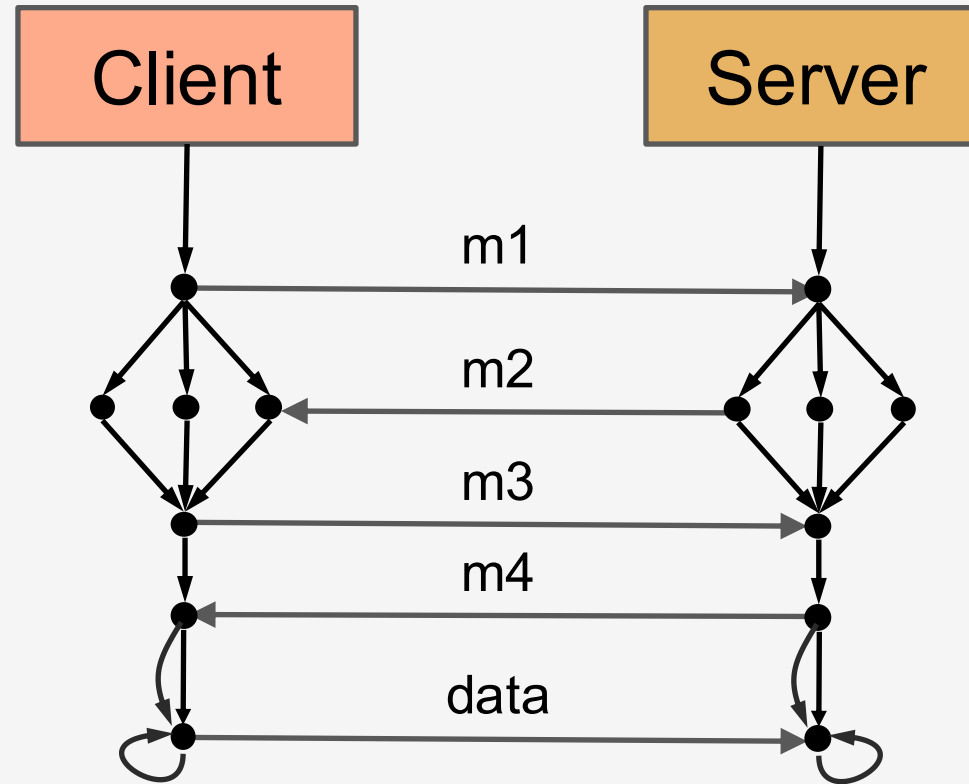


We use security protocols daily



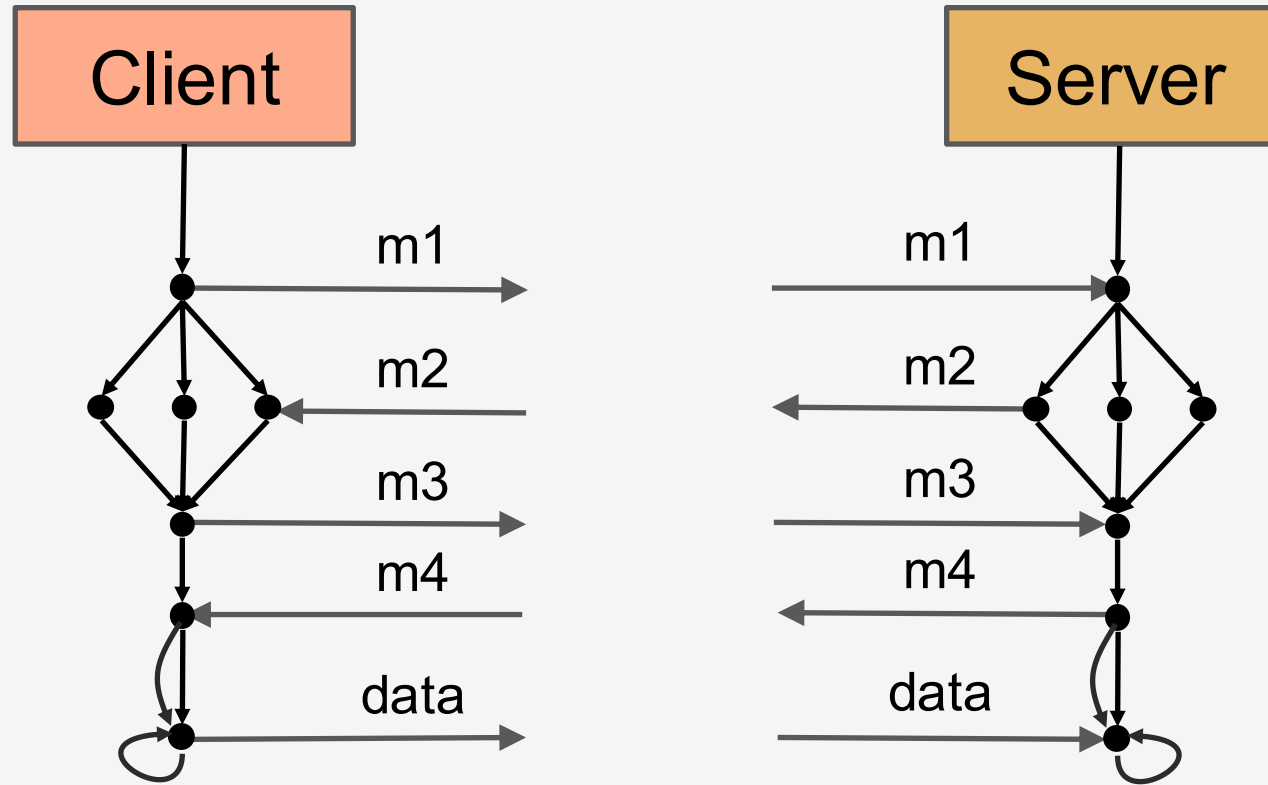


Security protocols and threat models



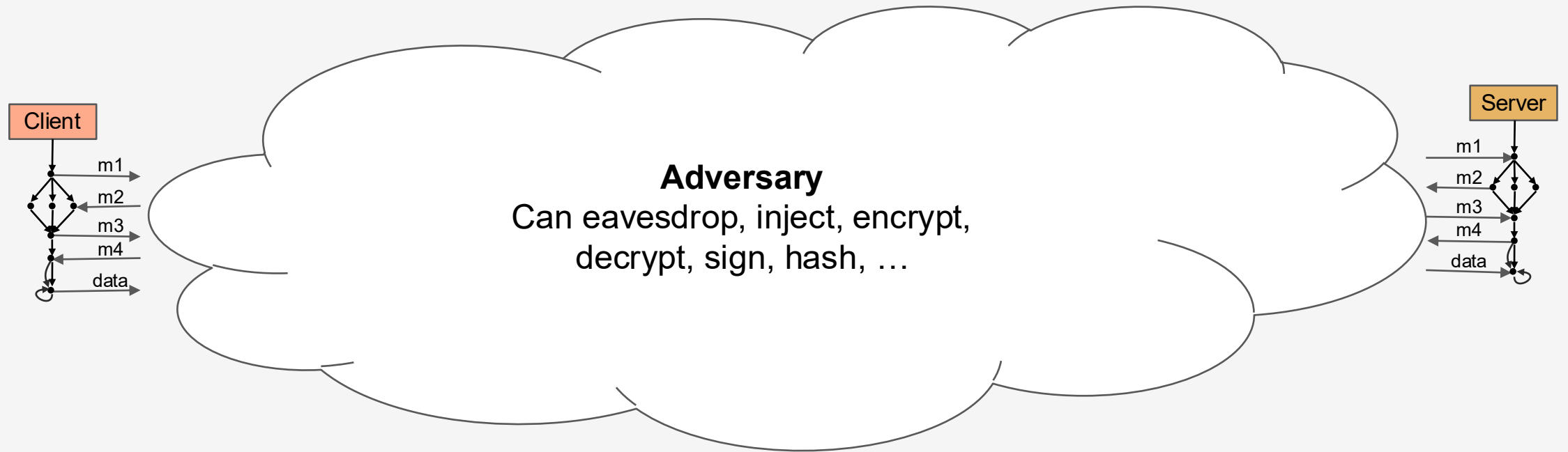


Security protocols and threat models



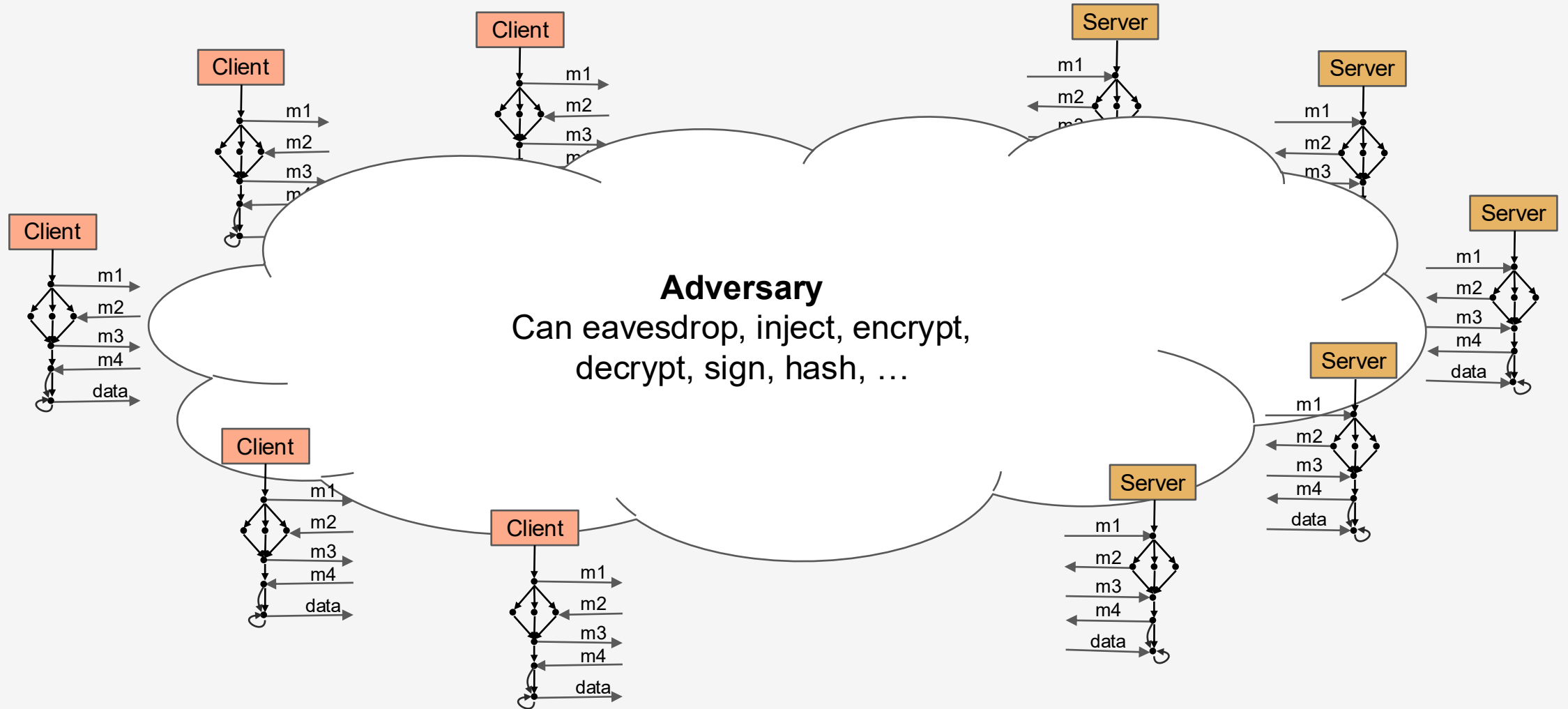


Security protocols and threat models



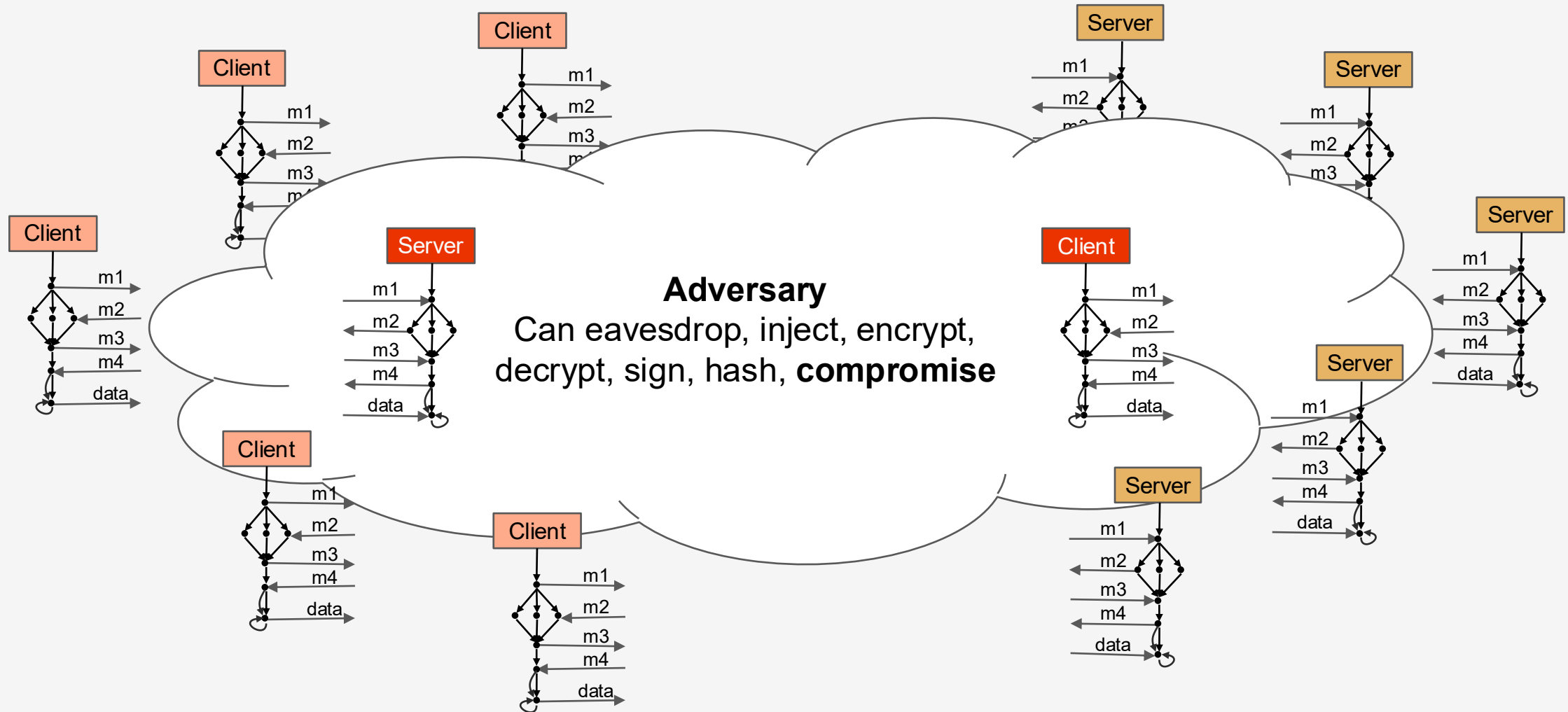


Security protocols and threat models



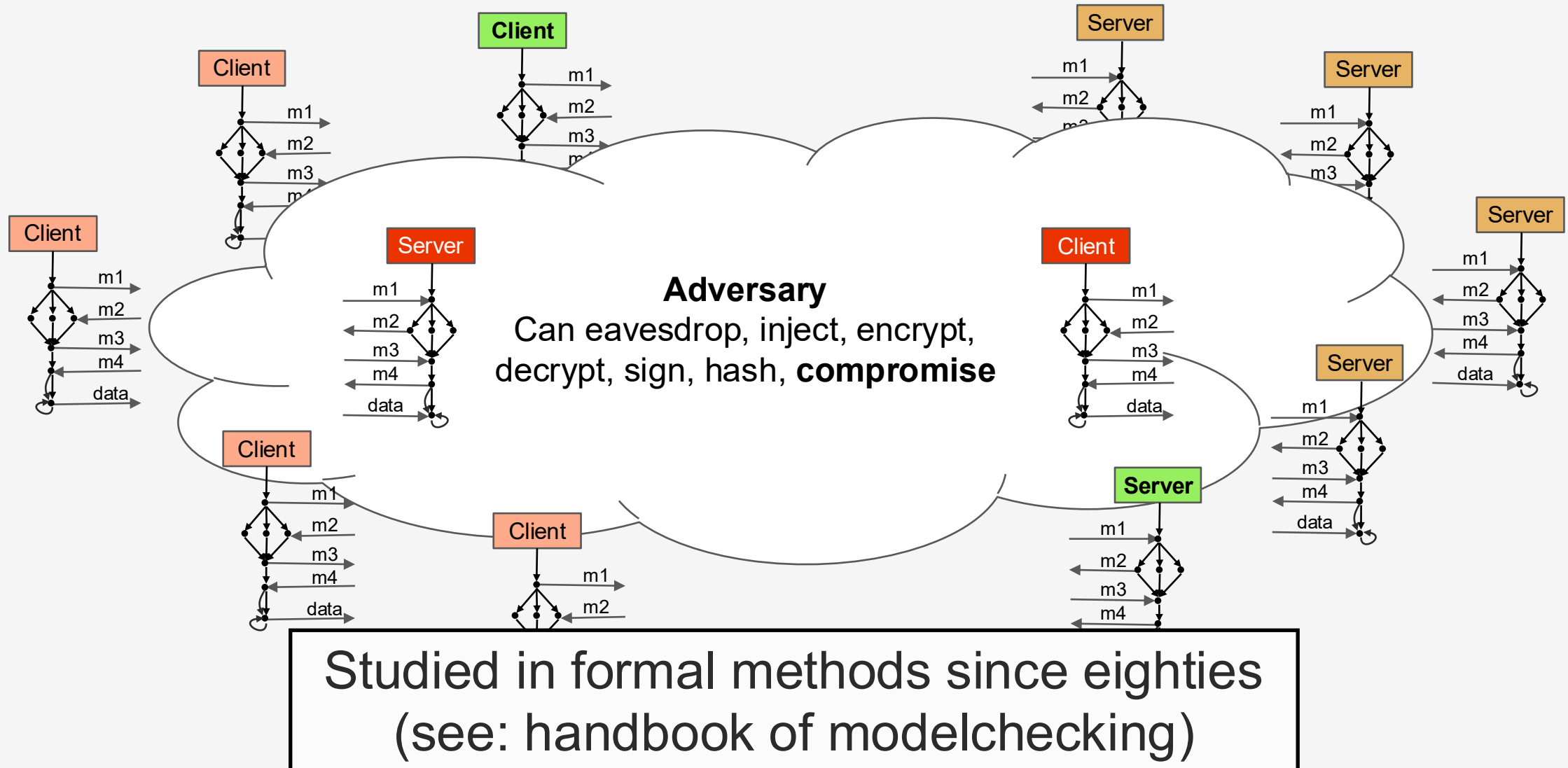


Security protocols and threat models





Security protocols and threat models





Tamarin prover



Theorem Prover

Constraint solver



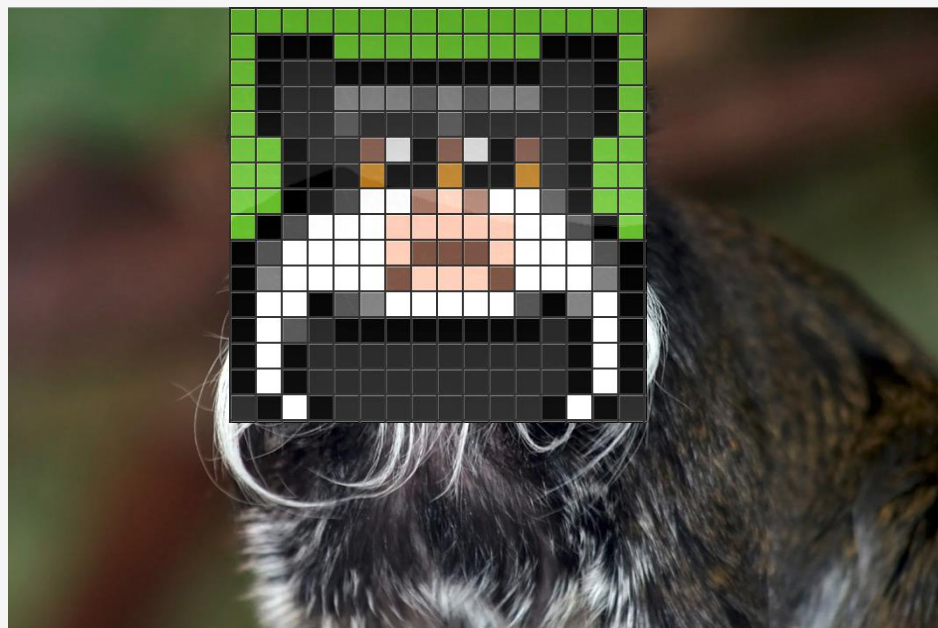
Tamarin prover



Emperor Tamarin



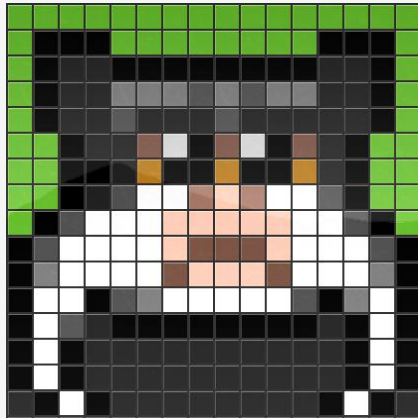
Tamarin prover



Emperor Tamarin



Tamarin prover



Development core:

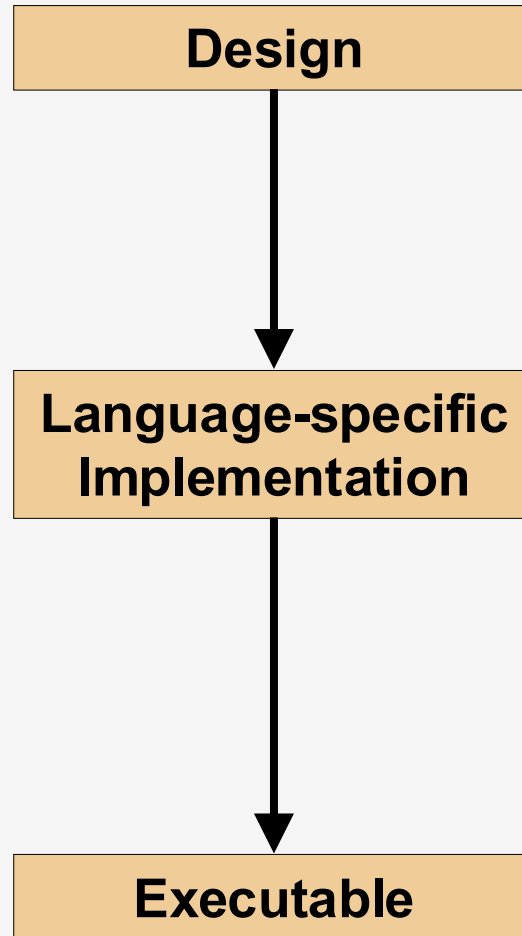
- CISP (Germany: Cremers)
- ETH Zurich (Switzerland: Basin, Sasse)
- LORIA (France: Dreier)

Open-source development

tamarin-prover.com

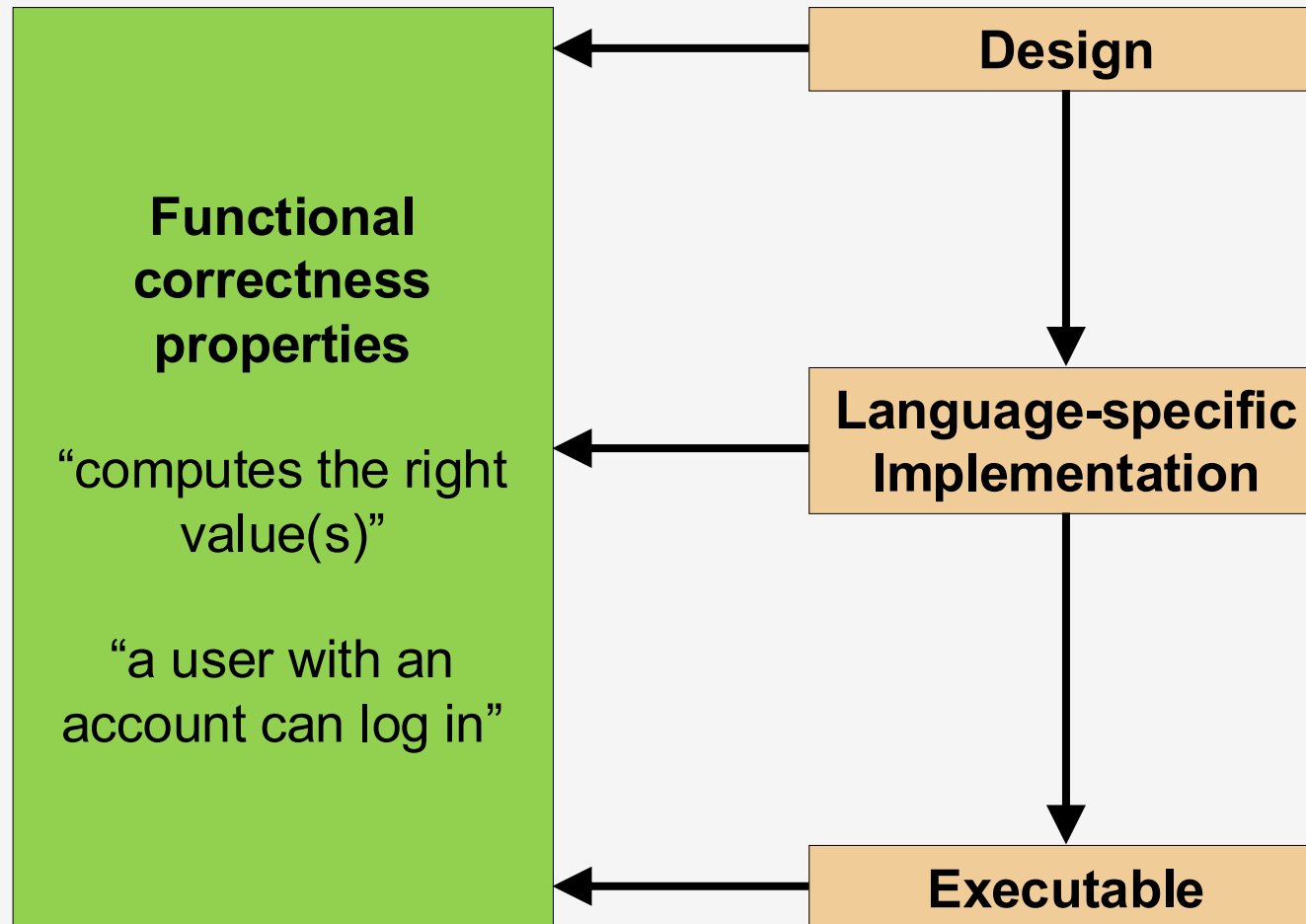


Application domains



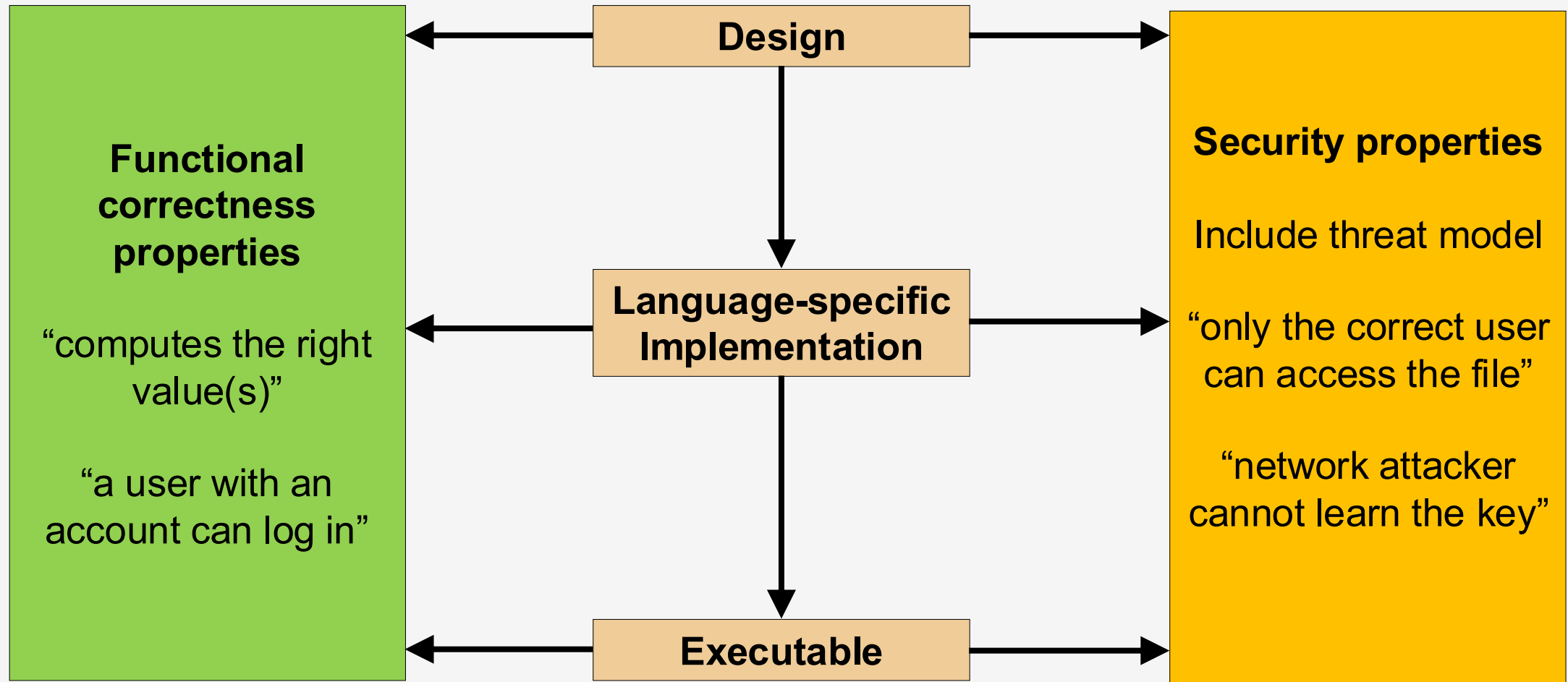


Application domains



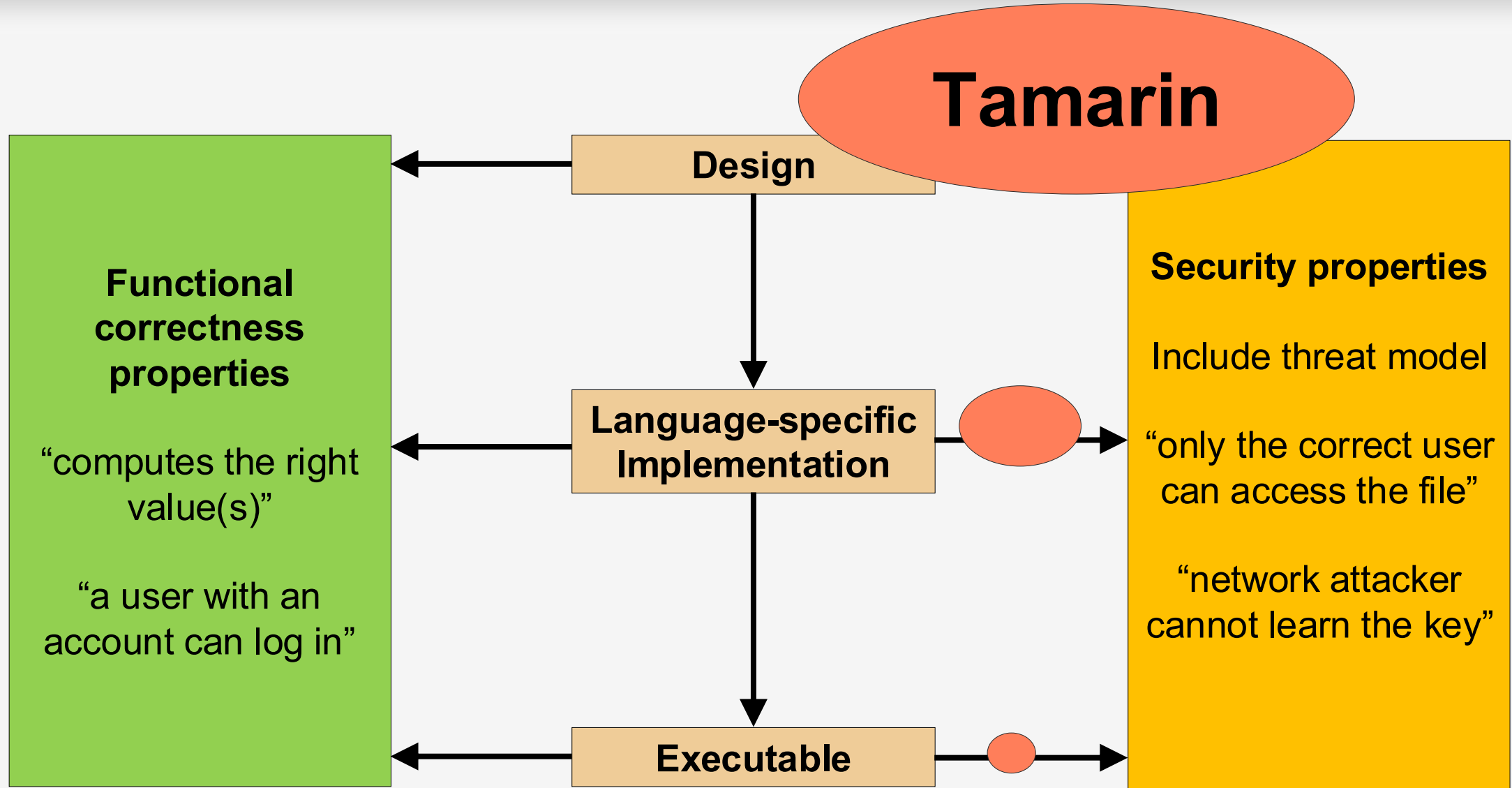


Application domains





Application domains





Tamarin prover: workflow

Internet Engineering Task Force (IETF)
Request for Comments: 8446
Obsoletes: 5077, 5246, 6961
Updates: 5705, 6066
Category: Standards Track
ISSN: 2070-1721

E. Rescorla
Mozilla
August 2018

The Transport Layer Security (TLS) Protocol Version 1.3

Abstract

This document specifies version 1.3 of the Transport Layer Security (TLS) protocol. TLS allows client/server applications to communicate over the Internet in a way that is designed to prevent eavesdropping, tampering, and message forgery.

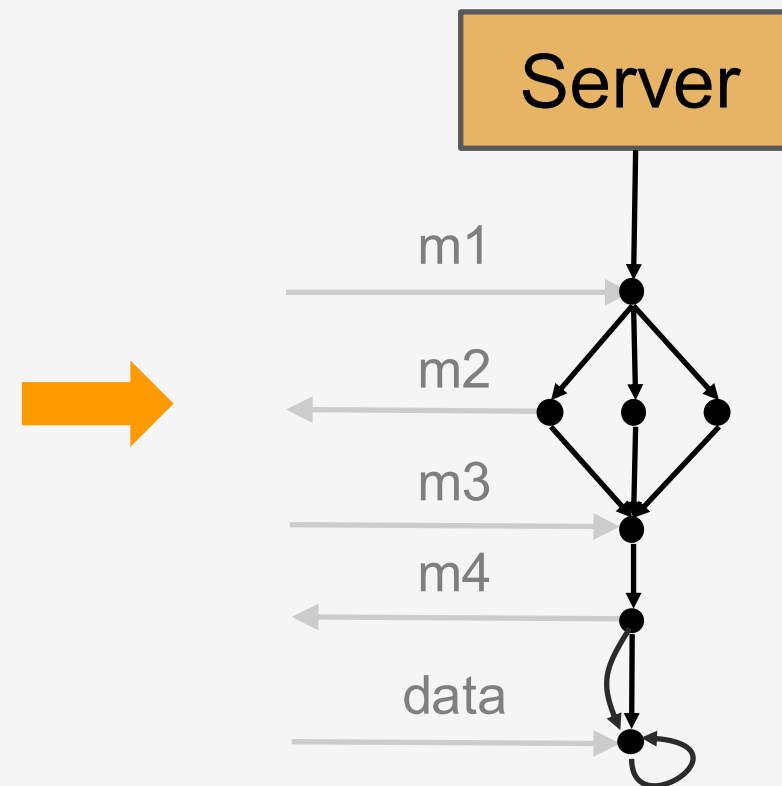
This document updates RFCs 5705 and 6066, and obsoletes RFCs 5077, 5246, and 6961. This document also specifies new requirements for TLS 1.2 implementations.

Status of This Memo

This is an Internet Standards Track document.

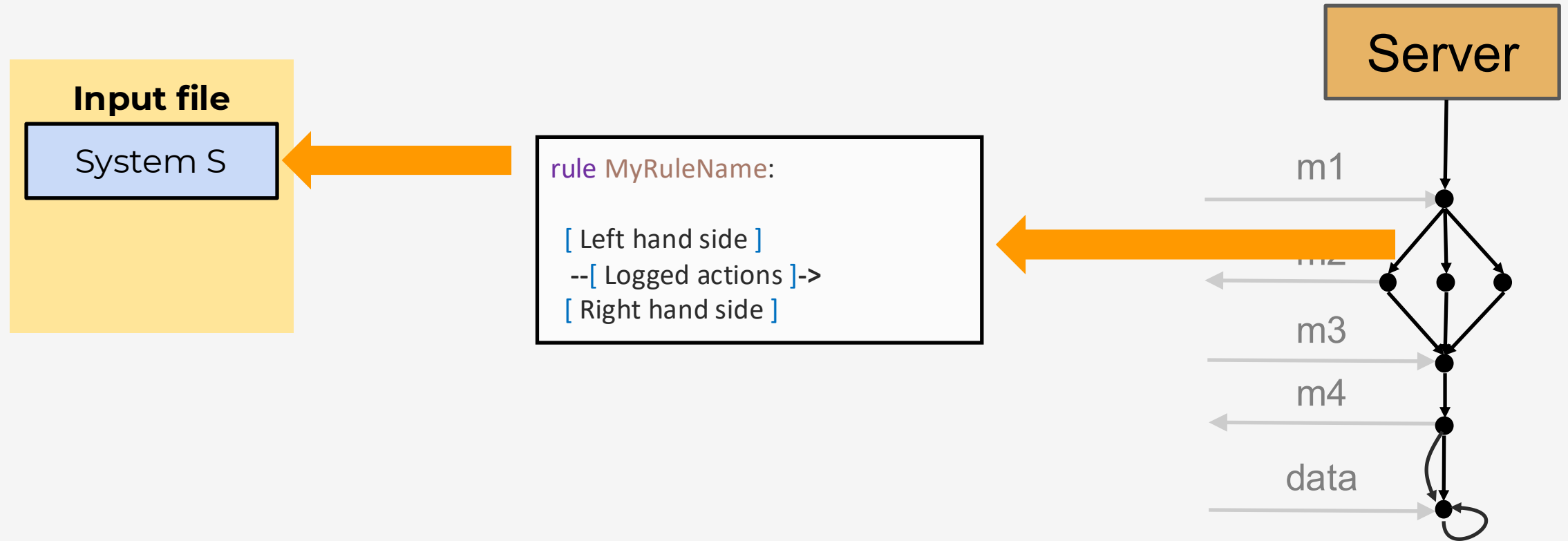
This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata,



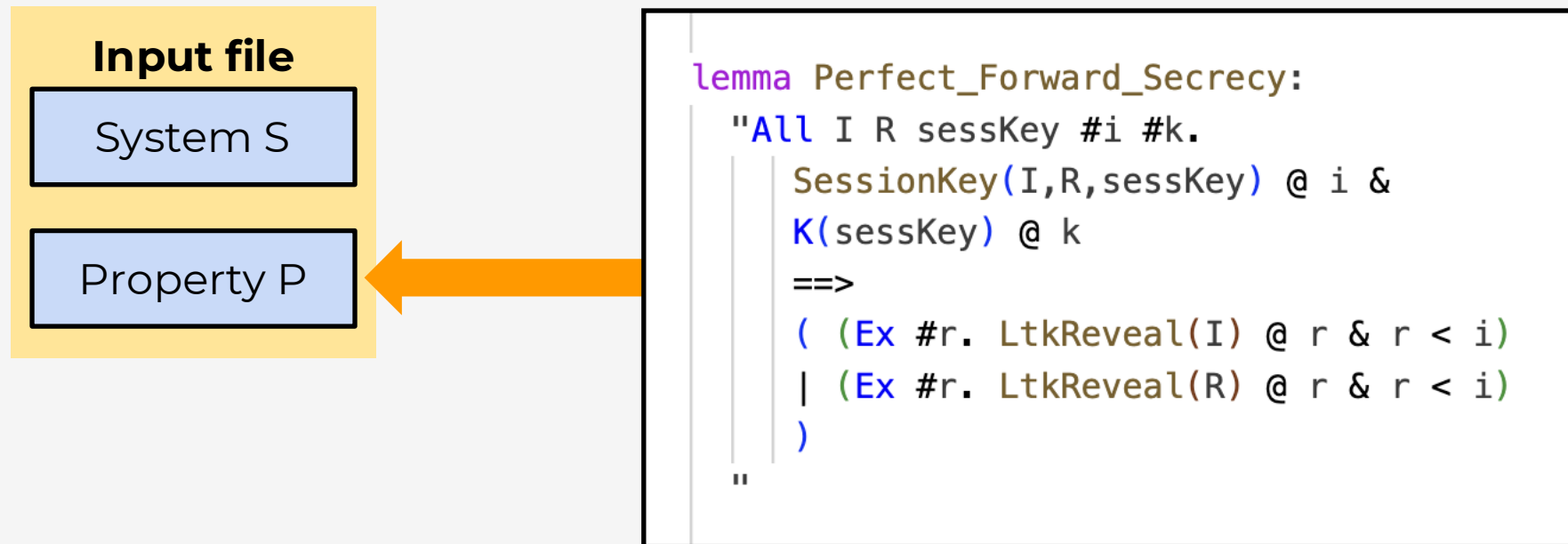


Tamarin prover: workflow



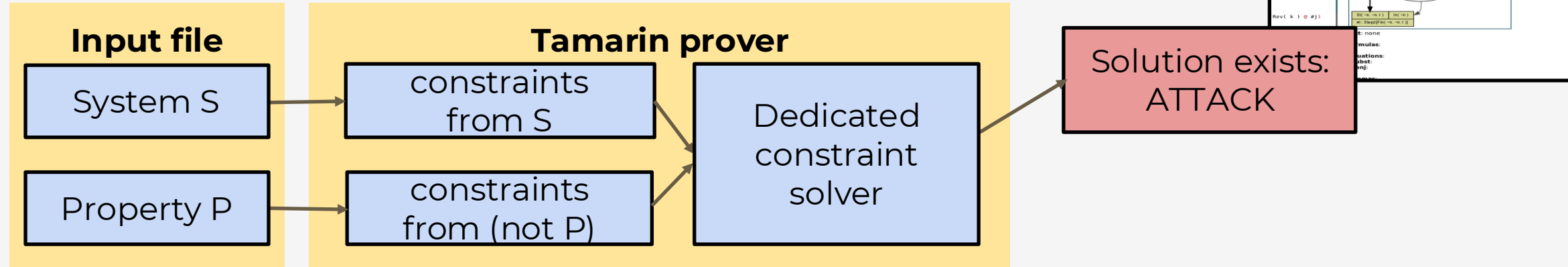


Tamarin prover: workflow





Tamarin prover: workflow



Visualization display

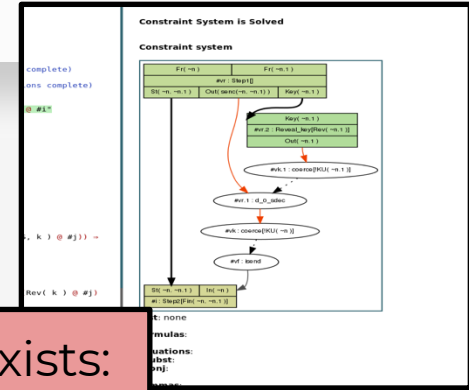
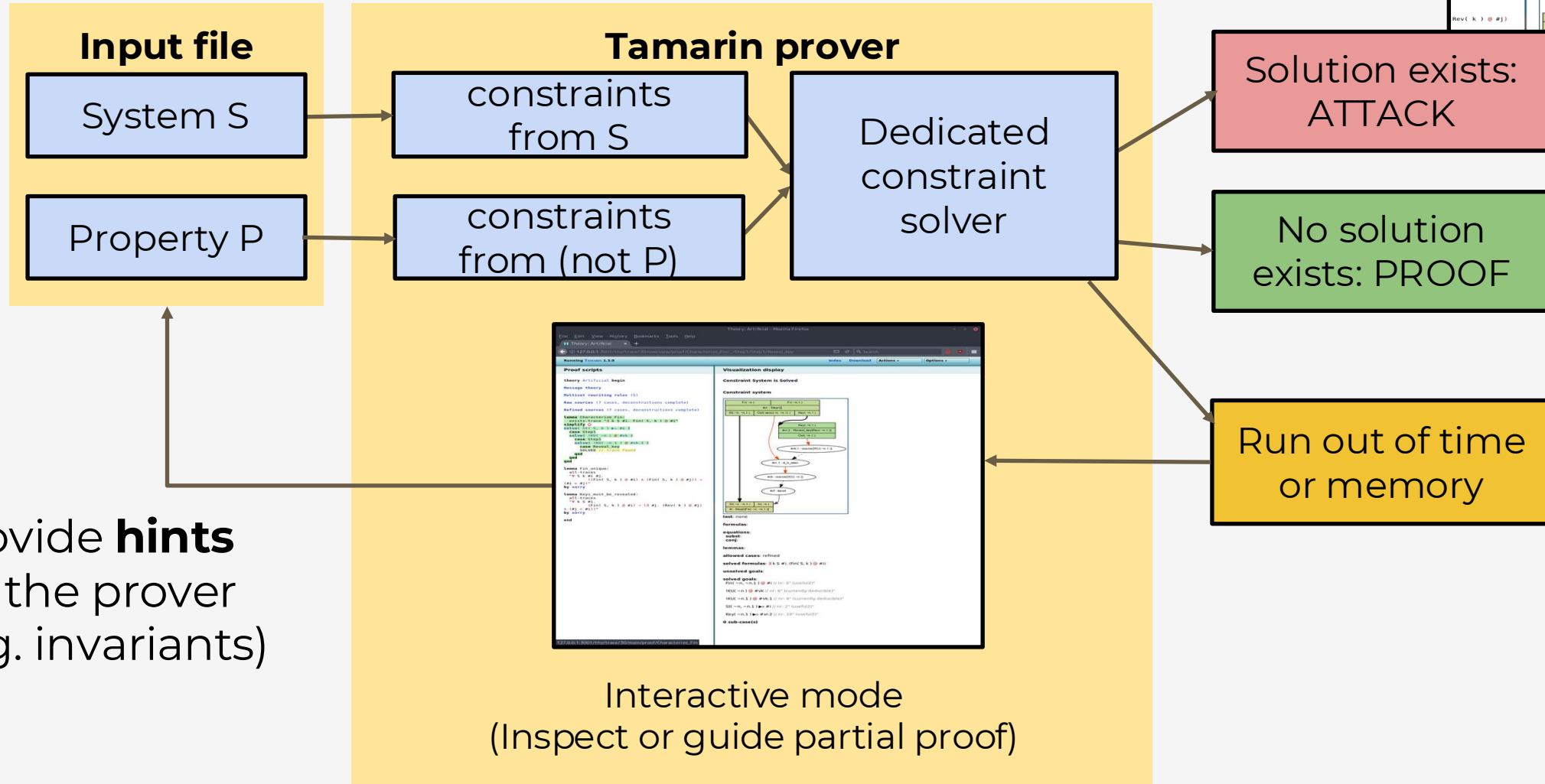
Constraint System is Solved

Constraint system

22



Tamarin prover: workflow





Innovation in techniques for deduction and symbolic reasoning

Tamarin's foundations

CSF 2012
CAV 2012

Operational semantics

multiset rewriting with equational theories

Property specification

fragment of first order logic with quantification over timepoints



Innovation in techniques for deduction and symbolic reasoning

Tamarin's foundations

CSF 2012
CAV 2012

S&P 2014
POST 2017
JCS 2022
CCS 2025 🏆

CCS 2015
CSF 2018
CSF 2023

Algorithmic developments



Innovation in techniques for deduction and symbolic reasoning

Tamarin's foundations

CSF 2012
CAV 2012

S&P 2014
POST 2017
JCS 2022
CCS 2025 🏆

CCS 2015
CSF 2018
CSF 2023

CCS 2019
CSF 2019 🏆
USENIX 2023 🏆 🏆
CCS 2024 🏆

Algorithmic developments

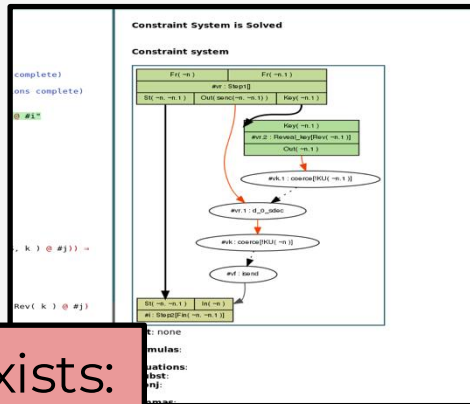
More precise handling of
cryptographic primitives



Innovation in techniques for deduction and symbolic reasoning

Dedicated constraint solving for undecidable problem, yet effective in practice

Algorithmic model checking for infinite-state systems



Sound: if terminates with **falsified**, provides concrete counterexample/attack

Complete: if terminates with **verified**, then no counterexample / attack exists in model

Theory advancement and engineered for practical use



State of the art: Attacks Tamarin can find

- Large attack scenarios (TLS 1.3 Rev 10: 18 messages)
- Cross-protocol attacks
- Unintended state machine transitions
- Downgrade attacks
- Nonce-reuse attacks (eg WiFi with AES-GCM)
- Invalid Curve Points
- Small order points
- DSKS attacks (Duplicate Signature Key Selection)
- Length extension attacks
- Maliciously generated keys
- Post-quantum secure protocols
- Hybrid schemes

These attack types are not hard-coded!

Tamarin finds these through systematic, accurate modeling of cryptography and protocols.



Success stories

[TLS] Revision 10: possible attack if client authentication is allowed during PSK

Sam Scott <sam.scott89@gmail.com> | Sat, 31 October 2015 11:19 UTC | [Show header](#)

Dear all,

We [1] are in the process of performing an automated symbolic analysis of the TLS 1.3 specification draft (revision 10) using the Tamarin prover [2],

While revisiting client authentication, we found what we believe is a new attack on client authentication either in the initial handshake, or with a post-handshake signature over the handshake hash, our Tamarin analysis finds an attack. The



 Security Research

Blog

iMessage PQ3

October 27, 2023

Advancing iMessage security: iMessage Contact Key Verification



CVE-2023-31127

SPDM (PCI Express)

OpenCVE > Vulnerabilities (CVE) > CVE-2023-31127

libspdm is a sample implementation that follows the DMTF SPDM specification in libspdm prior to version 2.3.1. If a device supports both DHE session and

CVSS v3 **8.8 HIGH**

8.8 /10



Tamarin use in industry



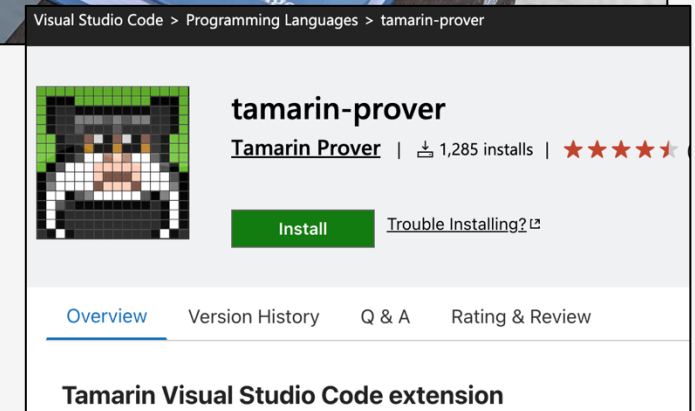
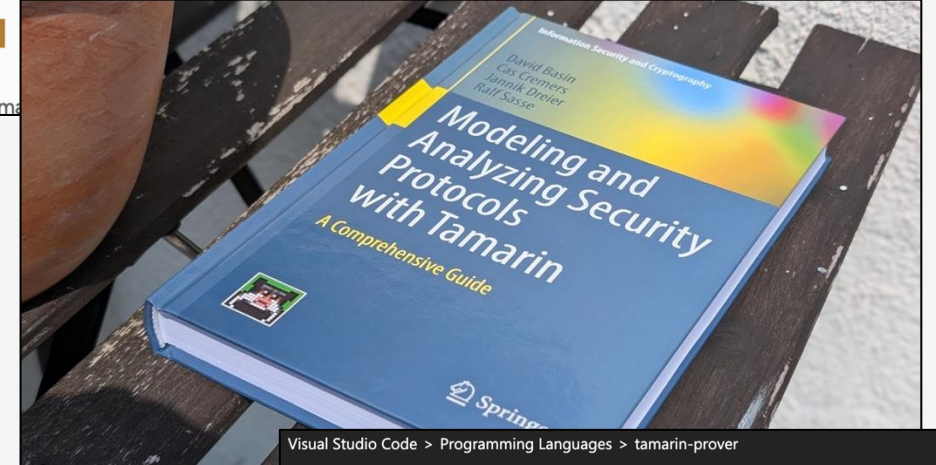
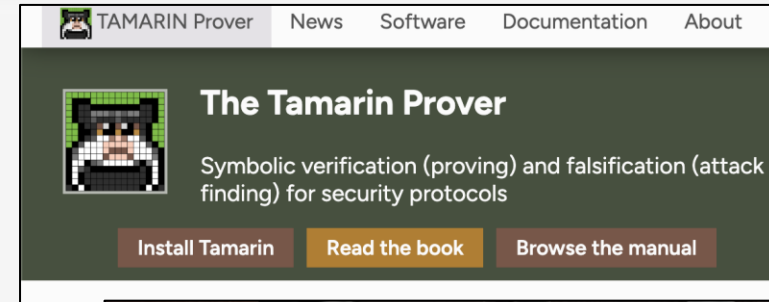


Maturity and development

Tamarin is a mature tool: you can use it now
tamarin-prover.com

- Open source development on github
- Active user community
- Teaching materials including book (free PDF)
- Many example protocol models
- Plugins for editors (e.g., VSC)

Under active development



Underlying model

Modeling in Tamarin

- Basic ingredients:
 - **Terms** (think “messages”)
 - **Facts** (think “sticky notes on the fridge”)
 - Special facts: **Fr(t)**, **In(t)**, **Out(t)**, **K(t)**
- State of system is a multiset of facts
 - **Initial state** is the empty multiset
 - **Rules** specify the transition rules (“moves”)



Modeling in Tamarin

- Basic ingredients:
 - **Terms** (think “messages”)
 - **Facts** (think “sticky notes on the fridge”)
 - Special facts: **Fr(t)**, **In(t)**, **Out(t)**, **K(t)**
- State of system is a multiset of facts
 - **Initial state** is the empty multiset
 - **Rules** specify the transition rules (“moves”)
- Rules are of the form:
 - $l \rightarrow r$
 - $l \rightarrow [a] r$



The model

- **Term algebra**
 - $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_ \^ _, _^{-1}, _ \^ _, 1, \dots$

The model

- **Term algebra**

- $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_ \^ _, _^{-1}, _ \^ _, 1, \dots$

- **Equational theory**

- $\text{dec}(\text{enc}(m, k), k) =_E m,$
 - $(x \wedge y) \wedge z =_E x \wedge (y * z),$
 - $(x^{-1})^{-1} =_E x, \dots$

The model

- **Term algebra**

- $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_ \^ _, _^{-1}, _ \^ _, 1, \dots$

- **Equational theory**

- $\text{dec}(\text{enc}(m, k), k) =_E m,$
- $(x \wedge y) \wedge z =_E x \wedge (y * z),$
- $(x^{-1})^{-1} =_E x, \dots$

- **Facts**

- $F(t_1, \dots, t_n)$

The model

- **Term algebra**

- $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_, _, _^{-1}, _*, 1, \dots$

- **Equational theory**

- $\text{dec}(\text{enc}(m, k), k) =_E m,$
- $(x^y)^z =_E x^{(y * z)},$
- $(x^{-1})^{-1} =_E x, \dots$

- **Facts**

- $F(t_1, \dots, t_n)$

- **Transition system**

- State: multiset of facts
- Rules: $\vdash [a] \rightarrow r$

The model

- **Term algebra**

- $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_, _, _^{-1}, _*, 1, \dots$

- **Equational theory**

- $\text{dec}(\text{enc}(m, k), k) =_E m,$
- $(x^y)^z =_E x^{(y * z)},$
- $(x^{-1})^{-1} =_E x, \dots$

- **Facts**

- $F(t_1, \dots, t_n)$

- **Transition system**

- State: multiset of facts
- Rules: $l \rightarrow [a] \rightarrow r$

- **Tamarin-specific**

- Built-in Dolev-Yao attacker rules: $\text{In}(\),$
 $\text{Out}(\), K(\)$

The model

- **Term algebra**

- $\text{enc}(_, _), \text{dec}(_, _), h(_, _),$
 $_, _, _, ^{-1}, _, ^*, 1, \dots$

- **Equational theory**

- $\text{dec}(\text{enc}(m, k), k) =_E m,$
- $(x^y)^z =_E x^{(y * z)},$
- $(x^{-1})^{-1} =_E x, \dots$

- **Facts**

- $F(t_1, \dots, t_n)$

- **Transition system**

- State: multiset of facts
- Rules: $l \rightarrow [a] \rightarrow r$

- **Tamarin-specific**

- Built-in Dolev-Yao attacker rules: $\text{In}(\),$
 $\text{Out}(\), K(\)$
- Special **Fresh** rule:
 - $[] \rightarrow [] \rightarrow [\text{Fr}(\mathbf{x})]$
 - Constraint on system such that $\mathbf{x}$ is unique

Semantics

- **Transition relation**

$S \xrightarrow{[a]}_R ((S \setminus^{\#} l) \cup^{\#} r)$, where

- $l \xrightarrow{[a]} r$ is a ground instance of a rule in R , and
- $l \subseteq^{\#} S$ wrt the equational theory

Semantics

- **Transition relation**

$S \xrightarrow{[a]}_R ((S \setminus^{\#} I) \cup^{\#} r)$, where

- $I \xrightarrow{[a]} r$ is a ground instance of a rule in R , and
- $I \subseteq^{\#} S$ wrt the equational theory

- **Executions**

$\text{Exec}(R) = \{ [] \xrightarrow{[a_1]} \dots \xrightarrow{[a_n]} S_n \mid \forall n. \text{Fr}(n) \text{ appears only once on right-hand side of rule} \}$

Semantics

- **Transition relation**

$S \xrightarrow{a}_R ((S \setminus I) \cup^\# r)$, where

- $I \xrightarrow{a} r$ is a ground instance of a rule in R , and
- $I \subseteq^\# S$ wrt the equational theory

- **Executions**

$\text{Exec}(R) = \{ [] \xrightarrow{a_1} \dots \xrightarrow{a_n} S_n \mid \forall n. \text{Fr}(n) \text{ appears only once on right-hand side of rule} \}$

- **Traces**

$\text{Traces}(R) = \{ [a_1, \dots, a_n] \mid [] \xrightarrow{a_1} \dots \xrightarrow{a_n} S_n \in \text{Exec}(R) \}$

Semantics: example 1

- **Rules**

- rule 1: $[] \rightarrow [A('5')]$
- rule 2: $[A(x)] \rightarrow [B(x)]$

Semantics: example 1

- **Rules**

- rule 1: $[] \rightarrow [A('5')]$
- rule 2: $[A(x)] \rightarrow [B(x)]$

- **Execution example**

- $[]$
- $\rightarrow [A('5')]$
- $\rightarrow [A('5'), A('5')]$
- $\rightarrow [A('5'), B('5')]$

Semantics: example 1

- **Rules**

- rule 1: $[] \rightarrow [\text{Init}()] \rightarrow [A('5')]$
- rule 2: $[A(x)] \rightarrow [\text{Step}(x)] \rightarrow [B(x)]$

- **Execution example**

- $[]$
- $[\text{Init}()] \rightarrow [A('5')]$
- $[\text{Init}()] \rightarrow [A('5'), A('5')]$
- $[\text{Step}('5')] \rightarrow [A('5'), B('5')]$

- **Corresponding trace:** $[\text{Init}(), \text{Init}(), \text{Step}('5')]$

Semantics: example 2 (persistent facts)

- **Rules**

- rule 1: [] $\rightarrow$ [!C('ok'), D('1')]
- rule 2: [!C(x), D(y)] $\rightarrow$ [D(h(y))]

Semantics: example 2 (persistent facts)

- **Rules**
 - rule 1: $[] \text{ --[Init()]} \rightarrow [\textcolor{red}{!}C('ok'), D('1')]$
 - rule 2: $[\textcolor{red}{!}C(x), D(y)] \text{ --[Step(x,y)]} \rightarrow [D(h(y))]$
- **Execution example**
 - $[]$
 - $\text{--[Init()]} \rightarrow [\textcolor{red}{!}C('ok'), D('1')]$
 - $\text{--[Step('ok','1')]} \rightarrow [\textcolor{red}{!}C('ok'), D(h('1'))]$
 - $\text{--[Step('ok',h('1'))]} \rightarrow [\textcolor{red}{!}C('ok'), D(h(h('1'))))]$

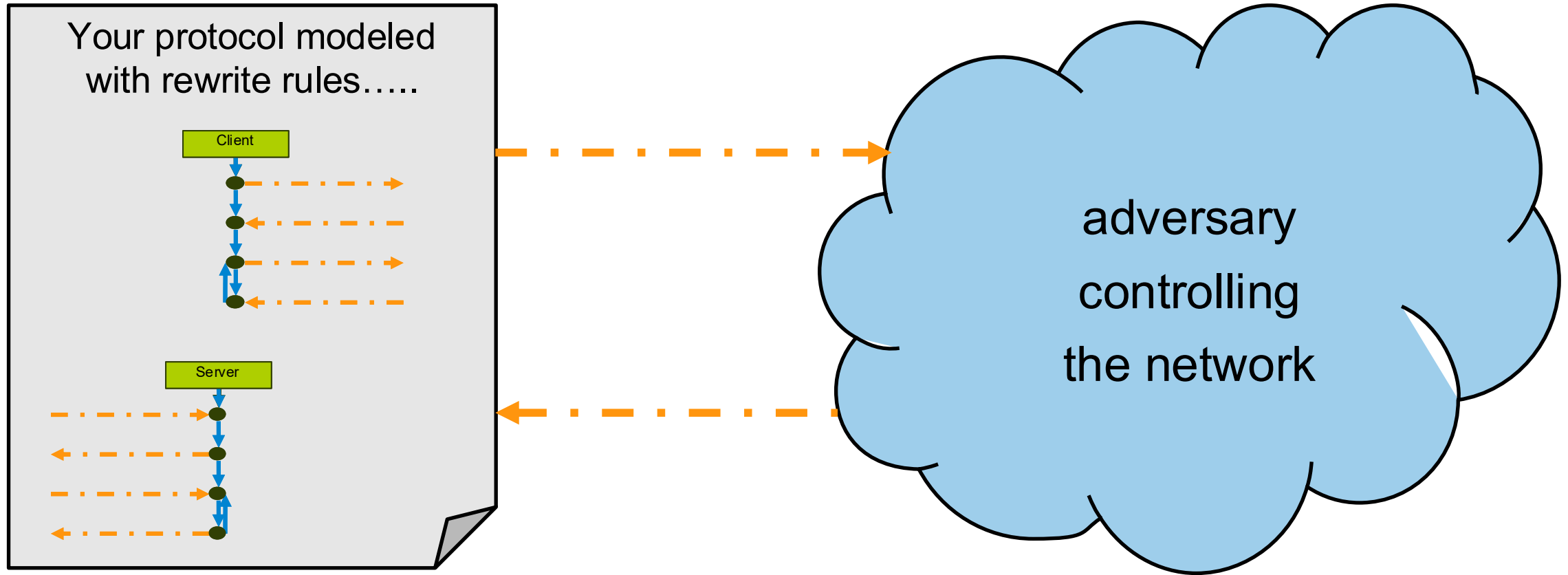
Semantics: example 2 (persistent facts)

- Rules
 - rule 1: $[] \rightarrow [!C('ok'), D('1')]$
 - rule 2: $[!C(x), D(y)] \rightarrow [D(h(y))]$
- Execution example
 - $[]$
 - $\rightarrow [!C('ok'), D('1')]$
 - $\rightarrow [!C('ok'), D(h('1'))]$
 - $\rightarrow [!C('ok'), D(h(h('1')))]$
- Corresponding trace: $[\text{Init}(), \text{Step}('ok', '1'), \text{Step}('ok', h('1'))]$

Semantics: adversary

- **Adversary-controlled network modeled by Out/In**
 - Adversary learns all terms x from each $\text{Out}(x)$
 - $\text{In}(x)$ can be instantiated with any x the adversary can construct
- **Adversary deduction**
 - Adversary can deduce new terms by applying functions and equational theories
 - Concretely: encrypt messages, hash, sign, decrypt when it knows the key, ...
- **Extending the adversary**
 - Concrete protocol models can introduce additional adversary capabilities
 - E.g. leaking (compromising) cryptographic keys

Tamarin tackles complex interaction with adversary



Tamarin tackles complex interaction with adversary

Your protocol modeled with rewrite rules

Client

Server

adversary
controlling
the network

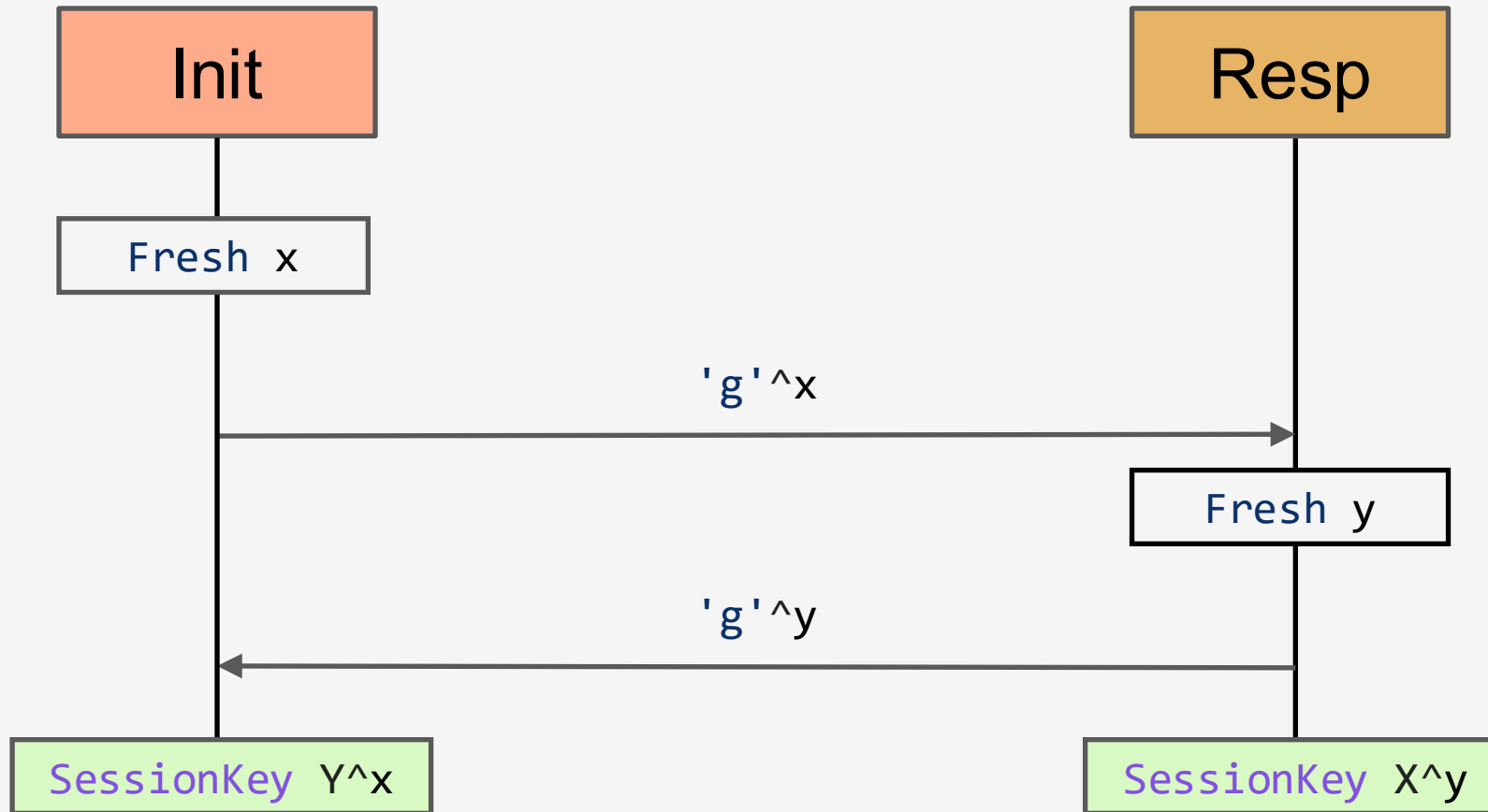
Specifying trace properties

- **Fragment of first-order logic with quantification over timepoints**
 - $F(t_1 \dots t_n) @ \#i$ $F(t_1 \dots t_n)$ appears at timepoint $\#i$ in the trace
 - Action facts on rules and $K(t)$ for adversary-derived terms
 - Logical connectives And, or, implication, negation
 - Equality Over terms, facts, or timepoints
 - Timepoint ordering $\#i < \#j$
 - Quantifiers Forall, Exists (All, Ex)
- **Universal and existential quantification over all traces of the system**
 - $P(t)$ holds for all traces t
 - There exists a trace t for which $P(t)$ holds

The Signed Diffie-Hellman protocol

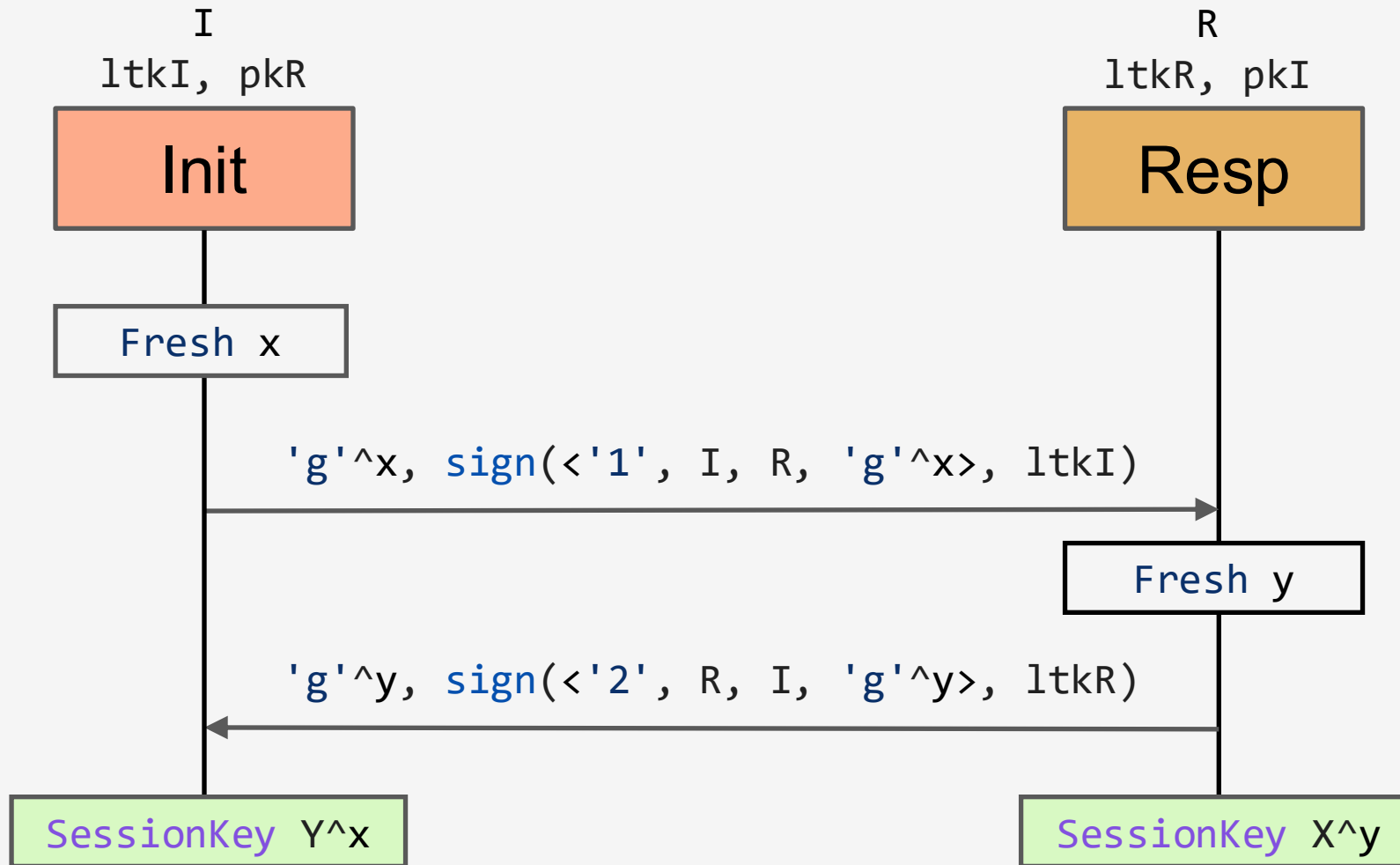


Diffie-Hellman Protocol





Signed Diffie-Hellman Protocol





Public Key Infrastructure (PKI)

'c'	constant
~x	fresh value
\$x	public value

```
rule Register_pk:
  [ Fr(~ltk) ]
-->
  [ !Ltk($A, ~ltk), !Pk($A, pk(~ltk)), Out(pk(~ltk)) ]
```



Responder

'c'	constant
~x	fresh value
\$x	public value

```
rule Resp:
  [ !Pk($I, pkI)
    , !Ltk($R, ltkR)
    , Fr(~y)
    , In( <$I, $R, X, m1> )
  ]
-->
  [ Out( <$R, $I, 'g'^~y, sign(<'2', $R, $I, 'g'^~y >, ltkR)> ) ]
```



Responder with signature verification

'c'	constant
~x	fresh value
\$x	public value

```
rule Resp:
  [ !Pk($I, pkI)
    , !Ltk($R, ltkR)
    , Fr(~y)
    , In( <$I, $R, X, m1> )
  ]
--[ _restrict( verify(m1, <'1', $I, $R, X>, pkI) = true ) ]->
  [ Out( <$R, $I, 'g'^~y, sign(<'2', $R, $I, 'g'^~y >, ltkR)> ) ]
```



Responder with session key

'c'	constant
$\sim x$	fresh value
$\$x$	public value

```
rule Resp:
  [ !Pk($I, pkI)
    , !Ltk($R, ltkR)
    , Fr(~y)
    , In( <$I, $R, X, m1> )
  ]
--[ _restrict( verify(m1, <'1', $I, $R, X>, pkI) = true )
  , SessionKey($I,$R, X^~y) ]->
  [ Out( <$R, $I, 'g'^~y, sign(<'2', $R, $I, 'g'^~y >, ltkR)> ) ]
```



Initiator step 1

'c'	constant
~x	fresh value
\$x	public value

```
rule Init_1:
  [ Fr(~tid), Fr(~x), !Ltk($I, ltkI) ]
-->
  [ Out( <$I, $R, 'g'^~x, sign(<'1', $I, $R, 'g'^~x>, ltkI) > ) ]
```



Initiator both steps

'c'	constant
$\sim x$	fresh value
$\$x$	public value

```
rule Init_1:
  [ Fr( $\sim$ tid), Fr( $\sim$ x), !Ltk( $\$$ I, ltkI) ]
-->
  [ Out( < $\$$ I,  $\$$ R, 'g' $^{\sim$ x}, sign(<'1',  $\$$ I,  $\$$ R, 'g' $^{\sim$ x>, ltkI) > ) ]
```

```
rule Init_2:
  [ !Pk( $\$$ R, pkR)
    , In( < $\$$ R,  $\$$ I, Y, m2> )
  ]
--[ _restrict( verify(m2, <'2',  $\$$ R,  $\$$ I, Y>, pkR) = true )
    , SessionKey( $\$$ I,  $\$$ R, Y $^{\sim$ x) ]->
  []
```



Initiator

'c'	constant
~x	fresh value
\$x	public value

```
rule Init_1:
  [ Fr(~tid), Fr(~x), !Ltk($I, ltkI) ]
-->
  [ Init_1( ~tid, $I, $R, ~x )
    , Out( <$I, $R, 'g'^~x, sign(<'1', $I, $R, 'g'^~x>, ltkI) > ) ]
```

```
rule Init_2:
  [ Init_1( ~tid, $I, $R, ~x )
    , !Pk($R, pkR)
    , In( <$R, $I, Y, m2> )
  ]
--[ _restrict( verify(m2, <'2', $R, $I, Y>, pkR) = true )
  , SessionKey($I, $R, Y^~x) ]->
  []
```



Security Property

lemma Secrecy:

```
"All I R sessKey #i.  
  SessionKey(I,R,sessKey)@i  
  ==> not (Ex #j. K(sessKey)@j) "
```

'c'	constant
~x	fresh value
\$x	public value
#x	timepoint



The protocol model

```
theory SignedDH_simple begin
builtins: diffie-hellman, signing

rule Register_pk:
  [ Fr(~ltk) ]
-->
  [ !Ltk($A, ~ltk), !Pk($A, pk(~ltk)), Out(pk(~ltk)) ]

rule Init_1:
  [ Fr(~tid), Fr(~x), !Ltk($I, ltkI) ]
-->
  [ Init_1( ~tid, $I, $R, ~x )
    , Out( <$I, $R, 'g'^~x,
          sign(<'1', $I, $R, 'g'^~x>, ltkI) > ) ]

rule Init_2:
  [ Init_1( ~tid, $I, $R, ~x )
    , !Pk($R, pkR)
    , In( <$R, $I, Y, m2> )
  ]
--[ _restrict(verify(m2, <'2', $R, $I, Y>, pkR) = true )
  , SessionKey($I,$R, Y^~x) ]->
[]
```

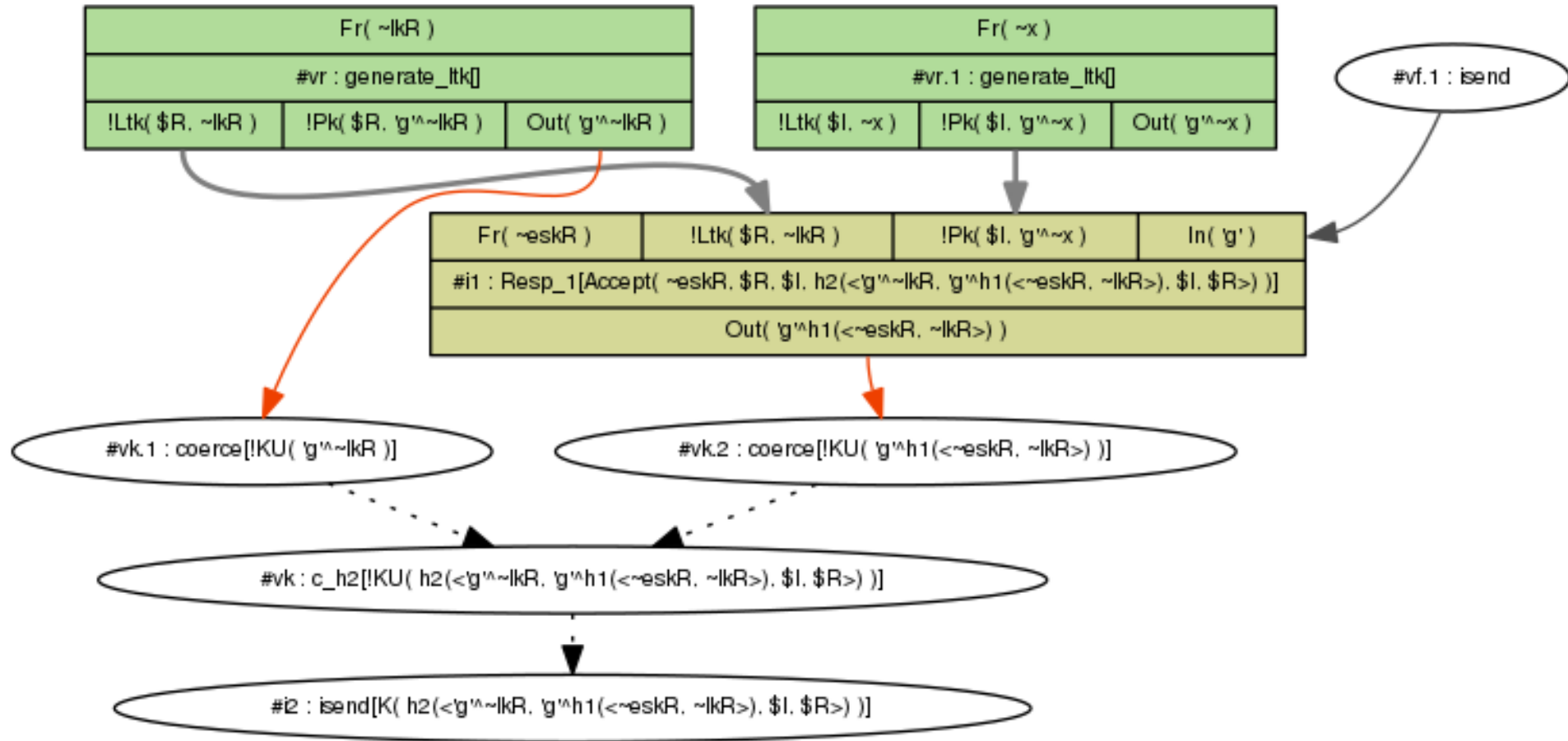
```
rule Resp:
  [ !Pk($I, pkI)
    , !Ltk($R, ltkR)
    , Fr(~y)
    , In( <$I, $R, X, m1> )
  ]
--[ _restrict(verify(m1, <'1', $I, $R, X>,
pkI) = true )
  , SessionKey($I,$R, X^~y) ]->
  [ Out( <$R, $I, 'g'^~y,
        sign(<'2', $R, $I, 'g'^~y >, ltkR)> ) ]

lemma Secrecy:
  "All I R sessKey #i.
   SessionKey(I,R, sessKey)@i
   ==> not (Ex #j. K(sessKey)@j) "

end
```

Demo

Reading Tamarin's graphs





Conclusions

- tamarin-prover.com
- Open source, well documented, and ready for use
- In preparation: Tamarin foundation
- Reach out to us or the Tamarin mailing list

Cas Cremers (cremers@cispa.de)

